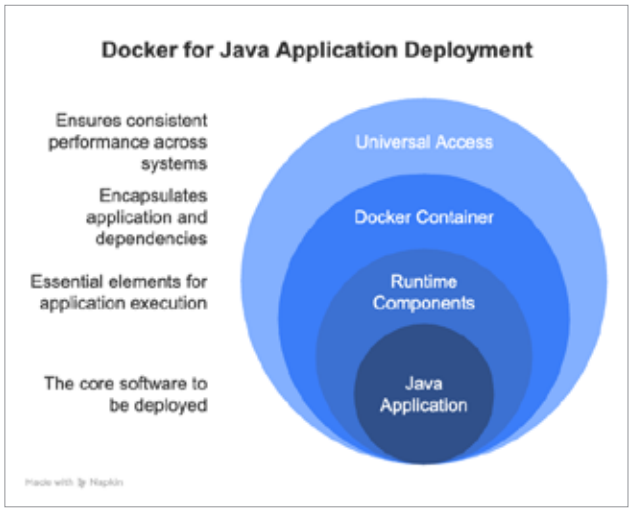

Figure 1: Java application deployment

Software development faces this issue frequently. Perfect code may operate without issues on your computer yet could encounter problems on different machines because the system configurations vary.

Docker functions as a container system for Java applications. A perfect analogy is delivering a complete meal inside a lunchbox instead of merely sharing a recipe for a sandwich. Docker creates a container that houses your app together with all the essential runtime components including Java code, configuration settings and the required files.

Any person who wants to launch your Java application needs to access your lunchbox. The application functions in an identical way for all users across all locations since it eliminates file loss and version discrepancies.

Here's an example. Imagine you've created a school project—a Java program that calculates grades. It runs perfectly on your laptop since you have Java 17 installed. However, when your teacher tries to run it on her computer with Java 11, it throws an error. Now, if you had used Docker, you could package your program along with Java 17 in a single container. This way, it wouldn't matter what version of Java your teacher has—your app would run smoothly because it carries its own Java environment.

Setting up your Java project

Before you dive into putting your Java application into a Docker container, it's essential to ensure that your project is well-prepared and neatly organised. Here's a step-by-step guide to help you get your Java project ready for Docker deployment.

Choose your Java application type: If you're working on a simple Java program, you may want to create a standalone JAR file. For web applications, particularly those using frameworks like Spring Boot, you'll usually build a runnable JAR that comes with an embedded web server. If you're dealing with older Java EE applications, you may have WAR files that require a separate application server. Understanding your project type is key to figuring out how to package it for Docker.

Table 1: Types of Java applications

Application type	Description	Example use case
Standalone app	Runs independently on a desktop or server	Grade calculator, PDF reader
Web application	Runs on a server and serves web pages over HTTP	Online banking system, e-commerce site
Microservice	Small, self-contained service in a larger system	Product catalogue in e-commerce

Table 2: Common Java packaging formats

File type	Full form	Used for	Runs on
.jar	Java ARchive	Standalone apps, libraries, Spring Boot	Java runtime (JRE/JDK)
.war	Web ARchive	Traditional web apps (servlets, JSPs)	Java EE servers (Tomcat, JBoss)
.ear	Enterprise Archive	Enterprise apps with multiple modules	Full Java EE servers

Table 3: Popular frameworks and their packaging

Framework / Style	Package type	How it runs
Plain Java (No framework)	.jar	Run via java -jar filename.jar
Spring Boot	.jar	Self-contained; has embedded server (Tomcat/Jetty)
Jakarta EE / Java EE	.war / .ear	Deployed on app servers like WildFly or GlassFish